**CampusNode LMS – Test Plan**

**Project:** CampusNode Learning Management System
**Course:** CS4433 – Software Engineering
**Testing Phase:** Phase 5 – Verification and Validation
**Testing Framework:** GoogleTest or Catch2 (to be integrated)
**Language:** C++17
**Database:** SQLite

## 1. Introduction

### 1.1 Purpose

This test plan defines the strategy for verifying and validating CampusNode LMS. Its purpose is to establish whether the implemented system satisfies the functional and non-functional requirements specified in the Software Requirements Specification (SRS).

Testing will cover individual business-logic components, interactions between components, complete system workflows, and acceptance of the system against stakeholder requirements.

### 1.2 Scope

The testing process will cover:

- Authentication and role-based access control.
- User, department and course management.
- Assignment creation and deadline management.
- Submission and resubmission.
- Late-submission penalties and extensions.
- Marking and written feedback.
- Moderation and release of marks.
- Audit logging.
- SQLite persistence and error handling.
- Offline operation and other applicable non-functional requirements.

### 1.3 Testing objectives

1. Verify that the system implements the Must-priority functional requirements.
2. Verify that each user role can only perform authorized operations.
3. Confirm that business rules BR1–BR8 are correctly enforced.
4. Verify that data is stored and retrieved correctly through the SQLite data-access layer.
5. Identify defects before the system is accepted.
6. Achieve at least 70% line coverage of the business-logic layer, as specified in the assignment.

## 2. Testing approach

The project follows the Waterfall development model, with verification and validation organized using the V-Model.

Testing will be performed at four levels.

| Testing level | Purpose | Main artefacts verified |
|---|---|---|
| Unit testing | Verify individual functions and classes independently. | Class design, business rules and individual service implementations. |
| Integration testing | Verify that connected components work together correctly. | Component architecture, repository interfaces, SQLite repositories and services. |
| System testing | Verify the complete application against its functional and non-functional requirements. | SRS and use-case specifications. |
| Acceptance testing | Determine whether the system meets stakeholder needs and is suitable for its intended use. | Stakeholder requirements and business rules. |

## 3. V-Model test mapping

| Development artefact | Corresponding verification activity |
|---|---|
| Stakeholder requirements | Acceptance testing |
| Software Requirements Specification | System testing |
| Architectural and behavioral design | Integration testing |
| Detailed class and component design | Unit testing |

Testing will proceed from individual components toward the complete application. Acceptance testing will then evaluate whether the system meets the agreed needs of its stakeholders.

## 4. Entry and exit criteria

### 4.1 Entry criteria

| Testing level | Entry criteria |
|---|---|
| Unit testing | Source code is available, required dependencies are installed, and the project can be configured and compiled. |
| Integration testing | Relevant unit tests have been implemented, and the components to be integrated are available. |

| | |
|---|---|
| System testing | The integrated application builds successfully, the database can be initialized, and the required user roles and test data are available. |
| Acceptance testing | System testing has been completed, critical defects have been resolved, and representative user workflows are available for demonstration. |

### 4.2 Exit criteria

| Testing level | Exit criteria |
|---|---|
| Unit testing | All planned unit tests have been executed, results recorded, and failures investigated. Business-logic line coverage target is at least 70%. |
| Integration testing | All planned component interaction tests have been executed, and critical integration defects are resolved. |
| System testing | All planned system scenarios have been executed, results recorded, and critical defects resolved or formally documented. |
| Acceptance testing | All critical stakeholder workflows have been demonstrated, results documented, and unresolved limitations reported. |

## 5. Test design techniques

### 5.1 Equivalence partitioning

Equivalence partitioning divides input data into groups expected to produce similar system behavior. A representative input from each group is selected for testing.

Examples include:

| Input | Equivalence classes | Expected behavior |
|---|---|---|
| Username | Existing username / non-existing username | Existing account is recognized; unknown account is rejected. |
| Password | Correct / incorrect password | Correct credentials permit login; incorrect credentials are rejected. |
| Mark | Within valid range / negative / above maximum | Valid marks are accepted; invalid marks are rejected. |
| Extension reason | Non-empty / empty | A valid reason is accepted; an empty reason is rejected. |
| User role | Authorized / unauthorized for an operation | Authorized operation succeeds; unauthorized operation is denied. |

### 5.2 Boundary value analysis

Boundary value analysis focuses on values at and around important limits.

The following boundaries are especially important in CampusNode LMS:

| Boundary | Test values | Expected behavior |
|---|---|---|
| Submission deadline | Just before, exactly at, and just after deadline | Correct on-time or late classification according to the implemented deadline comparison and specified business rule. |
| Late penalty | 0 days, 1 day, 2 days, 3 days and more than 3 days late | 0%, 10%, 20%, 30% penalty and zero mark after the three-day limit. |
| Maximum mark | 0, maximum allowed mark and a value above maximum | Valid range accepted; values outside the permitted range rejected. |
| Resubmission | Before, exactly at and after effective deadline | Resubmission permitted only while the effective deadline has not passed. |
| Extension | Later deadline, same deadline and earlier deadline | Only a valid later deadline is accepted. |

For deadline tests, the exact time comparison will be aligned with the implemented business rule and use-case specification.

## 6. Functional test coverage

The following areas will be covered by unit, integration and system testing:

| Test area | Requirements and business rules |
|---|---|
| Authentication | FR1 and applicable access-control requirements |
| User and course administration | FR2 |
| Assignment creation | FR3 |
| Course and assignment viewing | FR4 |
| Submission and late work | FR5, BR2 and BR3 |
| Resubmission | FR6 and BR5 |
| Extensions | FR7 and BR4 |
| Late penalties | FR8 and BR2 |
| Marking and feedback | FR9 and applicable marking rules |
| Moderation and release | FR10, FR11 and BR1, BR6 |
| Audit logging | FR12, FR13 and BR7 |
| Password management | FR14 |
| Database backup and restoration | NFR11 |

The final test-case identifiers and requirement mappings will be recorded in the test-case register and traceability matrix.

## 7. Non-functional testing

The following non-functional properties will be considered:

- **Performance:** Measure login response with 800 learner records stored, against the SRS target.
- **Reliability:** Verify that database errors do not crash the application or leave partially completed mark changes.
- **Availability:** Confirm that the system works without an internet connection.
- **Usability:** Evaluate whether representative lecturers can perform common tasks using the documented interface.
- **Portability:** Verify builds on the specified Windows and Ubuntu environments where available.
- **Maintainability:** Measure business-logic line coverage and inspect separation between the UI, services and data-access layers.
- **Recoverability:** Verify database backup and restoration.

Actual measurements and results will be recorded during test execution.

## 8. Test environment

The testing environment will use:

- C++17 compiler.
- CMake build system.
- SQLite database.
- Libsodium dependency.
- GoogleTest or Catch2 automated testing framework.
- A coverage measurement tool compatible with the selected compiler.
- GitHub repository for source-code and test-version management.

Testing will use a separate test database to avoid modifying real project data.

## 9. Defect management

Every identified defect will be recorded with:

- Defect ID.
- Description.
- Severity.
- Steps to reproduce.

- Expected result.
- Actual result.
- Root cause, when identified.
- Corrective action.
- Retest result.

Defects will be classified as Critical, Major, Moderate or Minor.

## 10. Test reporting

The final verification and validation report will contain:

1. Test plan.
2. Unit and integration test cases and source code.
3. Automated test execution results.
4. Business-logic line coverage report.
5. System and acceptance test results.
6. Defect log.
7. Summary of unresolved defects and limitations.

No test will be reported as passed until it has actually been executed and its result recorded.